Analysis of the lvgl embedded library

December the 20th, 2024

Elsa LHAYANI[1], Henri MAO[1], Louis-Marie MATTHEWS[1].

1. **Groupe OCC2**, ESILV, 12 avenue Léonard de Vinci, 92000 Courbevoie, France

Abstract

Embedded devices are ubiquitous, and society's reliance on them is widely expected to greatly increase. This poses serious concern as the code that is used to control these devices is often poorly checked and can contain bugs, software vulnerabilities or even backdoors. In this paper, we choose a well-known embedded system library and analyse its code using SonarQube, a commercial static analysis tool. The results indicate passable code quality in general but with a few notable exceptions.

Introduction

Embedded systems are becoming an integral part of many industries. They are extensively used in the military, in the police force, in industry, in agriculture and in customer services. The public at large is becoming increasingly reliant on them, either because they interact with embedded systems directly, or because some of the services they interact with depend on embedded systems.

This unprecedented rise in popularity comes from the additional liberties offerred by embedded systems. They replace mechanical, electrical and analogical systems because they possess microcontrollers. This makes them a versatile and powerful alternative. Water heaters for instance, which used to be purely electrical devices with some clever mechanical design, are now commonly found using embedded systems to regulate water temperature.

Another powerful feature of most embedded systems is the ability to connect to the Internet. (They are then called IoT devices.) The number of IoT devices alone is predicted to reach 32.1 billion devices in 2030. (Statista, 2024)

However, *usually* for cost-related reasons, the software that runs on those devices is of dubious quality. By being susceptible to vulnerabilities, IoT devices are used as support vector for large scale DDOS. (*DDoS Attacks And The Internet Of Things | DDoS-Guard*, 2023) Security cameras and other equipments can be hacked for other reasons, resulting in a loss of privacy, confidentiality, integrity and availability. More recently, a report by the US Commerce, Defense and Justice Departments accused TP-Link of voluntarily including vulnerabilities in its routers (which equip 60% of the US) as well as to sell its devices for less than the production cost. (Supan, 2024)

Checking the software that runs on embedded systems is not trivial however. The code is usually low-level, making it harder to read and understand, especially by third-parties and the open-source community. Besides, because the code runs on dedicated hardware, it can be very hard and costly–if not impossible–to test it, such as by using dynamic analysis. Virtualizers such as QEMU or Renode do exist, but they generally lack the features required for extensive testing. For these reasons, static analysis, which does not require running the code and can easily be added to most projects, can play a vital role in Securing Embedded Software™.

LVGL

The objective of this practical work is to analyze the embedded library "Light and Versatile Graphics Library" using SonarQube and to evaluate its quality and security.

LVGL is a libary that is used for creating graphical user interfaces on embedded systems. It is mature as it is compatible with a wide range of vendors and platforms, ranging from the ESP32 to the STM32, as well as Arduino and Adafruit.

We decided to analyse its code for the following reasons:

1. LVGL is a very popular embedded software that appeared as the 4th most starred Git project in the "embedded" category. As such, it is a good representative of the libraries that can be used on embedded systems. Its popularity also makes the findings of this study more useful and relevant to embedded system developpers.
2. LVGL is written mainly in C, with some minor parts in C++ and other languages.
3. LVGL is **fully-portable** with no external dependencies. Dependency management in C and C++ is often very tedious, and many mature C/C++ libraries are designed this way. This will make our analysis easier to set up.

Analysis

First, a fork of the library was made on GitHub. Afterwards, the repository was linked with SonarQube Cloud. This allowed to us to run a complete analysis of the library from SonarQube Cloud.

The results of the analysis can be divided into three broad categories:

- Security Hotspots
- Reliability Issues
- Maintainability Issues

Overall, SonarQube found 23 Reliability Issues, and more than 1,900 Issues related to Maintainability. There were also 10.8% of code duplication, and 42 Security Hotspots.

Security (Hotspots)

Several security hotspots were revealed during the static analysis of LVGL by SonarQube. These are categorized into high, medium and low priority issues with all types of vulnerabilities including Buffer Overflow and Denial of Service (DoS). They do not mean that the analyzed code is vulnerable to the mentionned vulnerabilities, but explicitely recommends checking against them nonetheless.

High Priority:

Buffer Overflow (30 issues)

Buffer Overflows can lead to a security vulnerability, for instance, giving the attacker access to sensitive data or allowing arbitrary code execution as we have seen during the attack lab TD where we used padding to find sensitive information. For instance, the issue “Make sure use of "`strcpy`" is safe here.” in `tests/src/test_cases/test_txt.c` (line 16):

```
strcpy(target, msg);
```

indicates that `msg` must be large enough to accomodate the content of `target`. Otherwise, a buffer overflow will occur, which can result in a program crash, memory leak, or exploitation by an attacker.

Medium Priority:

Denial of Service

Regular expressions are easy to use and very useful when searching for specific patterns, but they are infamous for their poor performance and are sometimes the cause of major outage. In the worst case, the complexity of the regular expression is exponential in the size of the input, this means that a small carefully crafted input (like 20 chars) can trigger catastrophic backtracking and cause a denial of service of the application. Super-linear regex complexity can lead to the same impact too with, in this case, a large carefully crafted input (thousands of chars).

Here is an example of some MicroPython code part of the analyzed library (`scripts/generate_lv_conf.py`, line 65):

```
groups = re.search(r'([A-Z0-9_]+)\s+(.+)', line).groups()
```

The issue is here: "Ex: Make sure the regex used here, which is vulnerable to polynomial runtime due to backtracking, cannot lead to denial of service." In other words, the analyzed project must ensure the input that is passed to the regex, and the regex itself, cannot require excessive computing power which may result in a software crash of the system or even premature use of its hardware components.

Permission issues

In Unix file system permissions, the "`others`" category refers to all users except the owner of the file system resource and the members of the group assigned to this resource.

Granting permissions to this category can lead to unintended access to files or directories that could allow attackers to obtain sensitive information, disrupt services or elevate privileges. For instance, we have "Ex: Granting permissions to "others" can lead to unauthorized access to files" in `tests/src/test_cases/test_file_explorer.c` (line 32):

```
mkdir("src/test_files/test_file_explorer_folder", 0777);
```

However, this code is only used for test cases, so production environments *shoud* not be affected, unless tests are shipped on the production hardware.

Low Priority:

Encryption of Sensitive Data

Because it does not rely on TLS, HTTP exposes applications to an extensive range of risks such as sensitive data exposure, traffic redirected to a malicious endpoint, malware-infected software update or installer, execution of client-side code or corruption of critical information.

The issue is here "Ex: Using http protocol is insecure. Use https instead." and we have it in `docs/_templates/layout.html` (line 24)

```
ga.src = ('https:' == document.location.protocol ? \
'https://ssl' : 'http://www') + \
'.google-analytics.com/ga.js';
```

This file is a template file used for the UI generated using lvgl. Restricting access to only HTTPS is essential.

No resource integrity

The issue "Ex: Make sure not using resource integrity feature is safe here" refers to `docs/_templates/layout.html` (line 6)

```
<script
    async src="https://www.googletagmanager.com/...">
</script>
```

Using remote artifacts without integrity checks can lead to the unexpected execution of malicious code in the application. On the client side, where front-end code is executed, malicious code could:

- impersonate users' identities and take advantage of their privileges on the application.
- add quiet malware that monitors users' session and capture sensitive secrets.
- gain access to sensitive clients' personal data.
- deface or otherwise affect the general availability of the application.
- mine cryptocurrencies in the background.

Those attacks are commonly referred to as XSS attacks. One-way to mitigate against this vulnerability is to use a CSP header.

Other Safety Concerns

The following is a small list of minor checklist items.

First, the issue "Ex: Make sure that sending signals is safe here." refers to the file `tests/micropy_test/__init__.py` (line 188):

```
os.killpg(os.getpgid(cls.process.pid), signal.SIGTERM)
```

Signaling processes or process groups can seriously affect the stability of this application or other applications on the same system. Accidentally setting an incorrect PID or signal or allowing untrusted sources to assign arbitrary values to these parameters may result in a denial of service. Also, the system treats the signal differently if the destination PID is less than or equal to 0. This different behavior may affect multiple processes with the same (E)UID simultaneously if the call is left uncontrolled.

Reliability

Reliability rules help to ensure the code behaves as expected. While a flagged issue does not necessarily mean the code will misbehave (*i.e.* is vulnerable or is dysfunctionnal), it generally indicates that the code does not conform to best practices that, if followed, could allow the static analysis tool (SonarQube in this case) to **guarantee** that the code is exempt of certain bugs or vulnerabilities.

Low-Impact Reliability Issues

One low-impact found issue is: "`"for" loop counters should not have essentially floating type (c:S2193)`. As its name indicates, it advises using an integer type when looping instead of a float, because floats are subject to rounding errors, even more so because rounding behavior depends on the compiler and the architecture. This means that while the offending code might work well during tests, it may be susceptible to bugs on another compiler or target.

```
static int test_cubic_bezier_ease_functions(...)
{
    int x1, y1, x2, y2, y;
    float t, t_step, fy;

    t_step = .001f;
    x1 = LV_BEZIER_VAL_FLOAT(fx1);
    y1 = LV_BEZIER_VAL_FLOAT(fy1);
    x2 = LV_BEZIER_VAL_FLOAT(fx2);
    y2 = LV_BEZIER_VAL_FLOAT(fy2);

    for(t = 0; t <= 1; t += t_step) {
        // ...
    }

    return 1;
}
```

Here however, this is precisely the use of the function: to test whether the float-rouding behavior works as expected and is within the set `ERROR_THRESHOLD`. We can consider this a false positive.

Medium-Impact Reliability Issues

Medium-Impact Reliability Issues are generally small code issues, usually easy to fix, that can make the code more reliable. For instance:

- Related "if/else if" statements should not have the same condition

Here, we have two identical conditions in different branches of the same `if/else if` statement. As it is, the second branch will never be reached.

```
if(*state == LV_TEXT_CMD_STATE_WAIT) { /*Start char*/
    *state = LV_TEXT_CMD_STATE_PAR;
    ret = true;
}
/*Other start char in parameter is escaped cmd. char*/
else if(*state == LV_TEXT_CMD_STATE_WAIT) {
    *state = LV_TEXT_CMD_STATE_PAR;
}
```

This code is misleading and unnecessary complex. It is also not efficient as the same condition is potentially checked twice.

- Integral operations should not overflow

We have the following code:

```
`value & (1 << (n_bits - 1))`
```

Here, `n_bits - 1` is potentially negative because `n_bits` is defined as an integer, and even more so because it `1` is substracted from it. As a result, the right operand can be negative, which makes the left shift operation undefined.

High-Impact Reliability Issues

The NULL concept is controversial, and is even regretted by its creator (Hoare, 2009). It is part of the C standard however, which can lead to an issue if a NULL pointer (a pointer with a value of `NULL`) is **dereferenced**. In our analyzed project, we find multiple such instances. For instance, in the function `lv_font_get_glyph_dsc`:

```
dsc_out->box_h = font_p->line_height;
```

Here, `font_p` is dereferenced, and its `line_height` property is accessed. However, `font_p` is only defined as a parameter of the function with the type `const lv_font_t * font_p`, so it might very well be a NULL pointer!

Blocking-Impact Reliability Issues

In this category, we find issues that need urgent fixing to increase the reliability of the code.

One major rule that is broken in some places is: Memory access should be explicitly bounded to prevent buffer overflows. Buffer overflows are the most well-known software vulnerability, and are considered of a very high severity. (OWASP, n.d.)

Here is a piece of offending code that was found:

```
for(i = 0; i < grad_dsc->stops_count; i++) {
    if(grad_dsc->stops[i].opa < LV_OPA_MAX) {
        info->res = LV_COVER_RES_NOT_COVER;
        return;
    }
}
```

While this code will not necessarily result in a buffer overflow, the compiler nor the static analysis tool can guarantee this will not happen. This is why the recommended way to iterate over an array (which needs to

be static) is to use a macro, magic values (not very recommended for other reasons) or, better yet, `sizeof`!

Another important rule that posess several violations is: [Dynamically allocated memory should be released (c:S3584)]

```
png_bytep * row_pointers = (png_bytep *) malloc(...);
```

This line is part of the `write_png_file` function in `unity_support.c`. However, at no point is `row_pointers` deallocated, resulting in a loss of memory in the heap.

The remaining issue is self-explanatory: [Resources should be closed (c:S2095)] (*e.g.* when reading a file or opening a socket).

Maintainability

Maintainability Issues do not directly affect the build, and by extension the way the program or the library behaves. It remains important to address them however, as they can have an important impact on the development process, potentially introducing bugs, security vulnerabilities, or inefficient code.

They can be divided into several subcategories:

"Low" Maintainability Issues

This subcategory includes several minor issues that, when addressed, can improve the developer's experience, reduce the time to understand the code and make it easier to work on the project.

Minor Maintainability Issues mostly consist in making the code more restrictive to ensure the developer's intentions are clear, as well as to give the compiler and static analysis tools the possibility to emit a warning or an error if the developer's stated intention is not respected.

One such rule is "Declare the variable "i" inside the loop" (c:S5955). This rule is part of a broader principle that states that variables should only be declared in the context in which they are used. This makes it easier to understand what they are needed for, it also reduces the risk of the variable being inadvertently accessed or modified, and it also ensures the variable will not be stored in memory when it is not used.

Let's see an example in the analyzed project where this rule is not followed:

```
void lv_demos_show_help(void)
{
    int i;

    // Code hidden for readability

    for(i = 0; i < demos_count; i++) {
        LV_LOG("     %s \n", demos_entry_info[i].name);
    }
}
```

Here, we can see that the variable `i` is declared at the beginning on the function, even though it is only used inside the loop. It would be better to directly write:

```
for(int i = 0; i < demos_count; i++) {
    LV_LOG("     %s \n", demos_entry_info[i].name);
}
```

Another closely-related rule is c:S5350: "Pointer and reference local variables should be "const" if the corresponding object is not modified". As for the previous rule, it is about specifying certain constraints to the code in some places to narrow down the freedom left to the rest of the code. By limiting the possibilities in the rest of the code, it makes it easier to ensure and to check the rest of the code conforms to the developer's intentions,

Other maintainability issues that we will not discuss here because they are self-evident are: left-over TODO blocks in comments that need to be addressed, structures with too many fields and functions with too many parameters (which can make them hard to use and understand).

Let's note however the presence of false positives. For instance, the code sometimes calls a function named `LV_ASSERT_OBJ(obj, MY_CLASS);`, and SonarQube detects it as an empty statement, when it is in fact a macro used by a Python script to generate documentation.

"Medium" Maintainability Issues

In this subcategory are rules that make the code easier to maintain and more conformant to standards. For instance, rule `c:S814` "Change type of bit field" specifies that certain types should not be used when defining bit fields, because their behaviour is implementation-defined. (The code's behavior will depend on the compiler, and not on the C standard.)

```
lv_blend_mode_t blend_mode : 3;
```

Let's note the existence of an interesting issue: "Local variables and member data should not be volatile c:S3687". Understanding this issue requires knowing what a volatile variable is. A volatile variable is a variable that is declared in-code as subject to be accessed from outside the code itself. This means the compiler cannot ensure the variable remains of the correct type, or of the correct value, and it makes any optimisation impossible. For this reason, SonarQube flags the use of the `volatile` keyword as an issue because of the implications it has on the code. However, when working with embedded systems at a low level, the use of volatile types is often necessary because in memory-mapped systems, they are used to control the peripherals of the CPU.

We also find other rules, such as that advising merging nested `if` statements, moving `#include` directives at the top of file to follow conventions (which makes the codebase easier to understand and to manage), removing unused assignments (for improved readability and a more self-documenting code), and other rules of the same vein.

"High" maintainability issues

In this subcategory we find many self-explanatory issues, such as the use of nested 3 layers-deep control statements,

One very important issue that deserves to be explicitely mentionned here is: "A cast shall not remove any const or volatile qualification from the type of a pointer or reference

c:S859”. As previously discussed, the use of the `const` qualifier has the benefit of making the code more predictable for the developer and the compiler. Casting away the `const` attribute of a variable is thus very counter-productive, and can make bugs especially hard to find if the developer (rightly) assumes a variable remains constant when it does not. Here is an example of an offending line of code in our analyzed project:

```
(lv_obj_t *)obj
```

Where `obj` was previously defined with `const`.

Another interesting issue is “Cognitive Complexity of functions should not be too high c:S3776”. SonarQube calculates a metric on functions to estimate their “cognitive complexity”, which can be loosely defined as how hard it is to think about the function or to summarize what it does. A low cognitive complexity is preferred, because it makes it easier for the developer to think about the code, helping them spare mental resources that can be allocated for maintaining code.

Other issues that do not warrant a detailled explanation are the use of `#undef` statements (which undefine code and make it especially unpredictable and hard to understand), the use of potentially `NULL` variables, as well as certain code style conventions (e.g. "default" clauses should be first or last.

"Blocking" maintainability issues

"Blocking" Maintainability Issues are issues requiring urgent fix, because of their potential to introduce bugs or security vulnerabilities that can be otherwise very hard to detect.

In our analyzed project, we only find one type of issue of this category: The right-hand operands of && and || should not contain side effects (c:S912).

We have multiple instances of offending code in relation to this issue. One such instance is:

```
(elements > 0) && (elements--)
```

In this case, it is not immediately obvious that the compiler will optimise this statement, not executing `elements--` if `elements > 0` is `true`. Besides, if this is really the intended behaviour, **it is code whose behavior relies on a compiler optimization**, which is not desirable as it makes the code hard to understand and to maintain.

Conclusion

SonarQube was efficient in finding many of potential or actual issues in the code of the analized project, whether they were related to bad coding practices (code smells), risks of bugs, or presented a more important security risk such as buffer overflows, permission issues and vulnerabilities to DoS. We realized that code clarity was also important for security, as it reduced the risk of making a mistake leading to vulnerabilities.

Even if the code didn’t have inherent security issues, which shows that it was reasonably secure, it is important to note the security hotspots to review to avoid potential risks. However, the high number of code smells (1.9k), especially high severity (710) and blocker ones (2) highlights the fact that the code could have been written in a cleaner and easier to understand way to improve maintainability and make modifications and maintenance easier.

The code does not show a good reliability score however, as many fixes could be made to avoid impacting the features of the project and its reliability. The "blocker" issues and the high-impact issues should be reviewed first as a priority to avoid security issues and improve stability of the system. The next recommended step would be to include SonarQube in the analyzed’s project CI process, and to set Quality Gates (code quality objectives that need to be met and cannot decrease).

Bibliography

1. Statista. (2024, 11 septembre). *Number of IoT connections worldwide 2022-2033*. https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/
2. *DDoS Attacks and the Internet of Things* | DDoS-Guard. (2023, 14 novembre). DDoS-GUARD. https://ddos-guard.net/blog/ddos-attacks-and-the-internet-of-things
3. Supan, J. (2024, 19 décembre). TP-Link Routers Could Be Banned In the Next Year, Affecting Nearly 65 % of Internet Users in the US. *CNET*. https://www.cnet.com/home/internet/tp-link-routers-could-be-banned-in-the-next-year-affecting-nearly-65-of-internet-users-in-the-us/
4. Hoare, T. (2009, août 25). Null references : The billion dollar mistake. *InfoQ*. https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/
5. OWASP. (n.d.). Buffer overflow. In *OWASP*. Retrieved December 20, 2024, from https://owasp.org/www-community/vulnerabilities/Buffer_Overflow

Appendice: Installing SonarQube Community locally

Installing SonarQube locally is a tedious task. The documentation is spread accross numerous pages: some on the company's blog, some on several distinct GitHub repositories, and others on the support website.

A SonarQube Server ecosystem for a C/C++ project is made up of three parts:

1. A Scanner, that builds and compiles the code using the Build Wrapper from SonarQube to generate data about the project, and then run the Scanner from the root directory of the project to generate and send the report.
2. A Server, that will set up a website on which the SonarQube user interface and the report will be accessible, and to which the Scanner will send the report.
3. A Database for the Server.

At first, I attempted analyzing the code of the `FreeRTOS` project. First, I created a `Dockerfile` to install the compilation target, the Build Wrapper, and the Scanner.

```
FROM gcc:latest

RUN apt update && apt full-upgrade -y

# Install compilation target
RUN apt install gcc-arm-none-eabi -y

# Install SonarQube Build Wrapper
WORKDIR /tmp
RUN wget https://sonarcloud.io/static/cpp/build-wrapper-linux-x86.zip
RUN unzip build-wrapper-linux-x86.zip -d /opt/
RUN rm build-wrapper-linux-x86.zip
RUN echo "export PATH=/opt/build-wrapper-linux-x86:\$PATH" >> ~/.profile

# Install SonarQube Scanner CLI
WORKDIR /tmp
RUN wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-6.2.1.4610-linux-x64.z
RUN unzip sonar-scanner-cli-6.2.1.4610-linux-x64.zip -d /opt/
RUN rm sonar-scanner-cli-6.2.1.4610-linux-x64.zip
RUN echo "export PATH=/opt/sonar-scanner-6.2.1.4610-linux-x64/bin:\$PATH" >> ~/.profile

# Set default directory
WORKDIR /root/FreeRTOS-analyzed

# Enable symbolic links in the repository
RUN git submodule update --init --recursive
```

This was part of a Compose project to run all three services of SonarQube:

```
services:
scanner:
    tty: true
    stdin_open: true
    build: sonarqube/scanner
    volumes:
    - .:/root/FreeRTOS-analyzed
    networks:
    - ${NETWORK_TYPE:-ipv4}
sonarqube:
    image: sonarqube:community
    hostname: sonarqube
    container_name: sonarqube
    read_only: true
    depends_on:
    db:
        condition: service_healthy
    environment:
    SONAR_JDBC_URL: ${SONAR_JDBC_URL}
    SONAR_JDBC_USERNAME: ${SONAR_JDBC_USERNAME}
    SONAR_JDBC_PASSWORD: ${SONAR_JDBC_PASSWORD}
    volumes:
    - sonarqube_data:/opt/sonarqube/data
    - sonarqube_extensions:/opt/sonarqube/extensions
    - sonarqube_logs:/opt/sonarqube/logs
    - sonarqube_temp:/opt/sonarqube/temp
    ports:
    - "9000:9000"
    networks:
    - ${NETWORK_TYPE:-ipv4}
```

```
db:
    image: postgres:15
    healthcheck:
    test: ["CMD-SHELL", "pg_isready"]
    interval: 10s
    timeout: 5s
    retries: 5
    hostname: postgresql
    container_name: postgresql
    environment:
    POSTGRES_USER: ${SONAR_JDBC_USERNAME}
    POSTGRES_PASSWORD: ${SONAR_JDBC_PASSWORD}
    POSTGRES_DB: ${SONAR_DB_NAME}
    volumes:
    - postgresql:/var/lib/postgresql
    - postgresql_data:/var/lib/postgresql/data
    networks:
    - ${NETWORK_TYPE:-ipv4}

volumes:
sonarqube_data:
sonarqube_temp:
sonarqube_extensions:
sonarqube_logs:
postgresql:
postgresql_data:

networks:
ipv4:
    driver: bridge
    enable_ipv6: false
dual:
    driver: bridge
    enable_ipv6: true
    ipam:
    config:
        - subnet: "192.168.2.0/24"
        gateway: "192.168.2.1"
        - subnet: "2001:db8:2::/64"
        gateway: "2001:db8:2::1"
```

Once all three services were running and ready, the analysis could be started by running the following **from the Scanner container**:

```
build-wrapper-linux-x86-64 --out-dir build_wrapper_output_directory make clean all
sonar-scanner
```

Once finished, the report is uploaded to the local SonarQube Server. However, the results could not be accessed because we were using the community edition for a C project.

> ⚠ 4667 unanalyzed C files were detected in this project during the last analysis. C cannot be analyzed with your current SonarQube edition. Please consider upgrading to Developer Edition to find Bugs, Code Smells, Vulnerabilities and Security Hotspots in this file.

So, setting-up SonarQube locally was effectively a dead-end, and we decided to use the SonarQube Cloud instead, which is entirely free for open-source projects.